TEMPORAL · ACTION SIZING

Counting Your Actions Before Cloud

How to measure billable Actions on a self-hosted Temporal Service

Temporal Cloud bills on Actions. A self-hosted Service does not show you an Action total the way Cloud does, but the number is already in your environment. This recipe walks through the two ways to pull it: reading the Service's own metric, or counting from exported Workflow histories. Both feed the same simple conversion to a monthly figure your Temporal team can turn into a Cloud estimate. It follows Temporal's own guidance for estimating Actions before a migration.

Before you start

Check your Temporal Server version and confirm whether you have Prometheus or Grafana scraping the cluster. Version decides accuracy: 1.17 and later emit the metric, and 1.22.3 and later report it accurately for billing, including Local Activity metering. If you scrape metrics, use Path A. If you do not, use Path B.

PATH A · PREFERRED

Read the Action metric from Prometheus

1

Confirm the Server version

Anything 1.22.3 or newer gives a billing-accurate count. Between 1.17 and 1.22.2 the metric is still useful for load sizing but runs low on Local Activities, so treat it as an approximation.

2

Total Actions over your window

Run this in Prometheus or Grafana to get the Action count over the last 30 days. The metric is a counter, so increase() is what gives you the delta across the window. This needs samples spanning the whole window, so if the Server restarted inside it or your metrics retention is shorter, the number runs low.

```
sum(increase(action{service_name="frontend"}[30d]))
```

3

Set the window end correctly

The [30d] in the query defines the range, and Grafana evaluates it at the dashboard end time. So set the end of the dashboard time range to the end of the period you are measuring. The start you drag to does not matter, only the end does. Leaving it at now while you meant last month gives the wrong answer. For a single Namespace, add the label.

```
sum(increase(action{service_name="frontend",
exported_namespace="default"}[30d]))
```

4

Capture the shape of the load

Grab mean and peak Actions per second per Namespace too. Peak APS informs your Namespace APS limits, and it is where elastic scaling pays off versus overprovisioning self-hosted for the spike. If exported_namespace returns nothing, try the label namespace instead. You can also script this sampling with temporal-server-actions-count.

```
sum(rate(action{service_name="frontend"}[1m]))
by (exported_namespace)
```

Using Datadog instead of Prometheus? The metric name differs. Grab ready-made widget queries from datadog-self-hosted-queries.

PATH B · NO METRICS

Count from exported Workflow histories

1

Export a representative history per Workflow Type

Pull one Event History for each distinct Workflow Type. A single run stands in for the type, and you scale by volume later. Download from the Web UI with the Download button, or from the CLI. The counter accepts either the object format or the plain array format that these produce.

```
temporal workflow show \
--workflow-id <id> \
--output json > history.json
```

2

Install and run the counter

Run the temporal-history-action-count tool against each history file. It is installed from source, not PyPI, so fetch and run it in one step with uvx. It prints the billable Actions in that run, for example a total of 7. Child Workflows and Local Activities are handled for you, billed at 2x and collapsed to one respectively.

```
uvx --from \
git+https://github.com/temporal-community/temporal-history-action-count \
temporal-billable history.json
```

3

Scale by volume, then add what history hides

Multiply each type's per-run count by how many of those Workflows run per month, then sum across types. Then add the Actions that never land in Event History, so the counter cannot see them: roughly one Action per Query, plus Activity Heartbeats that reach the server. If you Query or Heartbeat heavily, leaving these out understates the total. See what counts as an Action for the full list.

THE MATH

Actions per second to monthly Actions

```
Monthly Actions = Mean APS × 60 × 60 × 24 × 30

Example: 95 APS × 2,592,000 = ~246M Actions / month
```

QUICK REFERENCE

Accuracy by Server version

Server version	What the metric gives you
`1.22.3 and later`	Billing-accurate, includes Local Activity metering
`1.17 to 1.22.2`	Useful for load sizing, runs low for billing
`Earlier than 1.17`	No action metric, use the Grafana dashboard workaround or Path B

Once you have a monthly Action total and your peak APS, share both with your Temporal Solutions Architect. The estimate is a starting point for a sizing conversation, not the final number.